

✓ PDS PRACTICAL 6

Name: Anushka kalidas Bhoyanwad

DIV: SY-A

Roll No: A-10

Aim: To implement web scraping in Python using the requests library and BeautifulSoup for extracting webpage content such as text, tags, attributes, and page title.

1) What is web scraping? Explain the role of the Requests library in web scraping?

Web scraping is the process of automatically collecting data from websites using Python programs.

Instead of manually copying content from a webpage, Python scripts extract information like headings, paragraphs, images, links, and tables.

The requests library is used to send an HTTP request to the target webpage and retrieve its HTML source code.

Example:

```
response = requests.get(url)
```

Here, the webpage content is fetched from the given URL.

2) What is BeautifulSoup? How does it help in parsing HTML?

Beautiful Soup is a Python library used to parse HTML and XML documents.

It converts the raw HTML source code into a structured format so that specific elements can be easily searched and extracted.

It helps in:

finding HTML tags extracting text reading attributes locating classes and ids
navigating webpage structure

Example:

```
soup = BeautifulSoup(response.text, 'html.parser')
```

3) Differentiate between web scraping and web crawling

Web Scraping	Web Crawling
Extracts specific data from websites	Visits and indexes webpages

Web Scraping	Web Crawling
Focuses on content extraction	Focuses on discovering links
Used for data analysis	Used by search engines
Example: extracting article text	Example: Google bot

4) What are the commonly used methods in BeautifulSoup?

Commonly used methods are:

find() → returns first matching element find_all() → returns all matching elements

get_text() → extracts text prettify() → formats HTML title → gets webpage title

Example:

```
soup.find_all("p") element.get_text(strip=True)
```

5) What are the ethical and legal considerations in web scraping?

Important considerations include:

follow website terms and conditions respect robots.txt avoid sending too many requests do not scrape personal/private data avoid copyright violations use data responsibly

```
import requests
from bs4 import BeautifulSoup

url = "https://example.com"
response = requests.get(url)

soup = BeautifulSoup(response.text, "html.parser")

print(soup.title.text)
```

Example Domain

```
import requests
from bs4 import BeautifulSoup

url = "https://example.com"
response = requests.get(url)

soup = BeautifulSoup(response.text, "html.parser")

paragraphs = soup.find_all("p")

for p in paragraphs:
    print(p.text)
```

This domain is for use in documentation examples without needing permission.
Learn more

```
import requests
from bs4 import BeautifulSoup

url = "https://example.com"
response = requests.get(url)

soup = BeautifulSoup(response.text, "html.parser")

links = soup.find_all("a")

for link in links:
    print(link.get("href"))
```

<https://iana.org/domains/example>

```
import requests
from bs4 import BeautifulSoup

url = "https://example.com"
response = requests.get(url)

soup = BeautifulSoup(response.text, "html.parser")

heading = soup.find("h1")
print(heading.text)
```

Example Domain

Conclusion:

In this assignment, web scraping was successfully implemented in Python using the requests library and BeautifulSoup. The requests library was used to fetch webpage content, while BeautifulSoup was used to parse and extract specific HTML elements such as titles, headings, paragraphs, and links. This assignment helped in understanding the basics of automated data extraction from websites along with the ethical and legal responsibilities involved in web scraping.

